

Convolutional Neural Network & Backpropagation Algorithm

Yunuo Cai

Content

- (Recall) Back Propagation(BP) Algorithm on Neural Network
- (Recall) Convolutional Network(CNN)
- BP Algorithm on CNN

Content

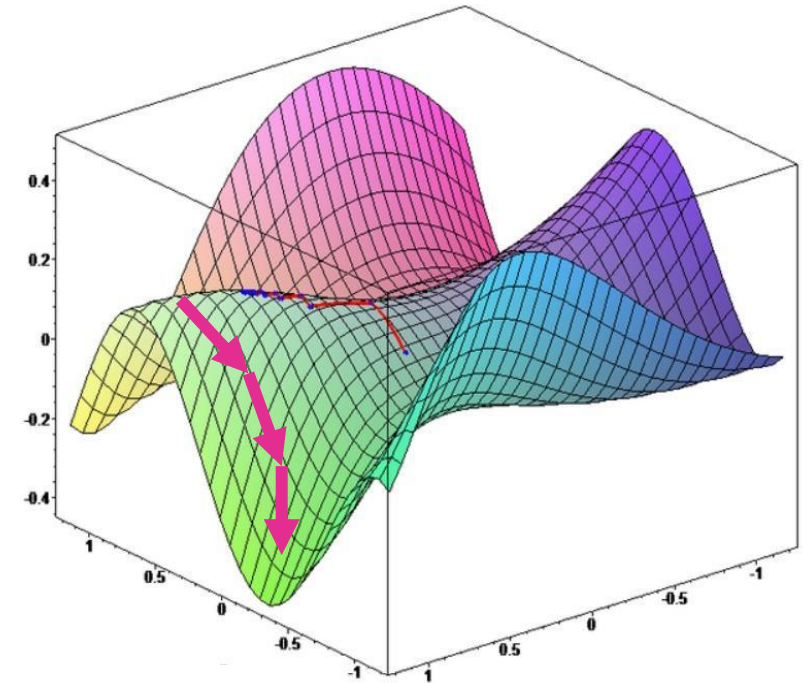
- **(Recall) Back Propagation(BP) Algorithm on Neural Network**
- (Recall) Convolutional Network(CNN)
- BP Algorithm on CNN

How to Optimize a Network h?

$$\operatorname{argmin}_{\theta} J(h(\theta | X), y)$$

- θ : Parameters
- X : Training Data (Fixed)
- J : A Loss function
- Gradient Descent:
 $\theta_{i+1} = \theta_i - a \cdot \nabla_{\theta} J(h(\theta_i | X), y)$ for $i = 0, 1, 2, \dots$

Like a person walking down the hills.

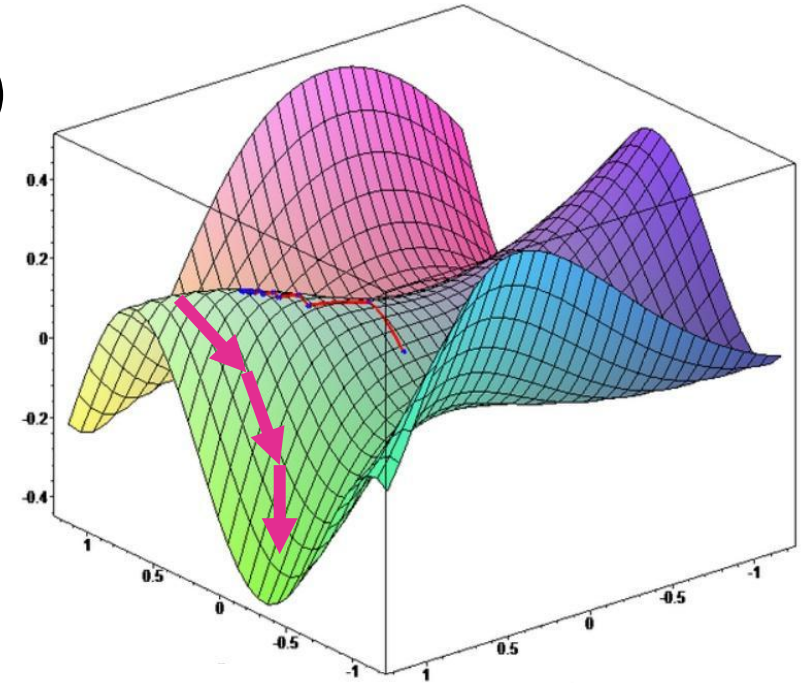


Gradient Descent

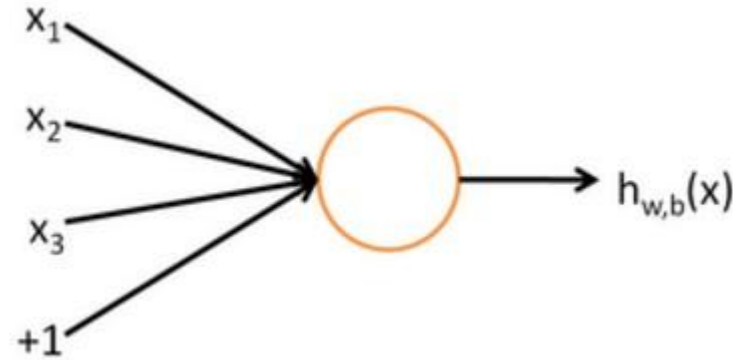
$$\operatorname{argmin}_{\theta} J(h(\theta | X), y)$$

- Compute $J(h(\theta_i | X), y)$ (Forward Propagation)
- Compute $\nabla_{\theta} J(h(\theta_i | X), y)$ (Backward Propagation)
- Update Parameters

Like a person walking down the hills.



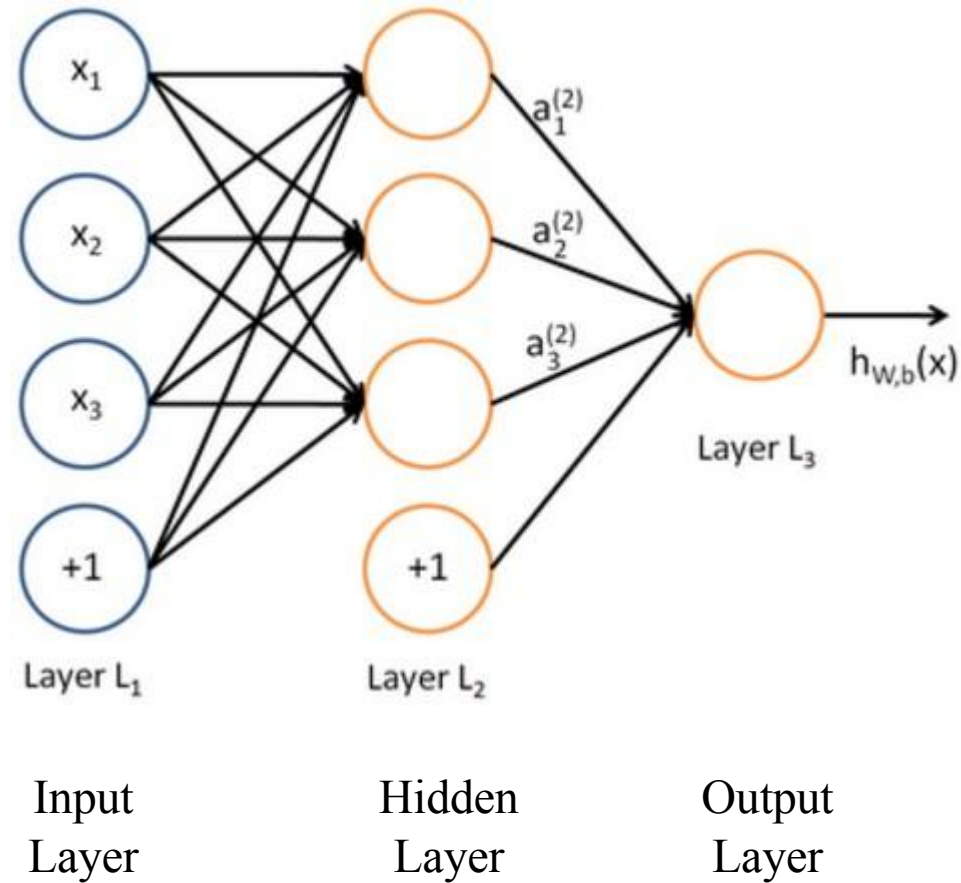
The Case in Neural Network — A Neuron



$$h_{W,b}(x) = f(Wx) = f(\sum_{i=1}^3 W_i x_i + b)$$

where $f : \mathbb{R} \mapsto \mathbb{R}$ is called the **activation function**

The Case in Neural Network — Multiple Neurons



The Case in Neural Network — Forward Propagation

n_l : the number of layers

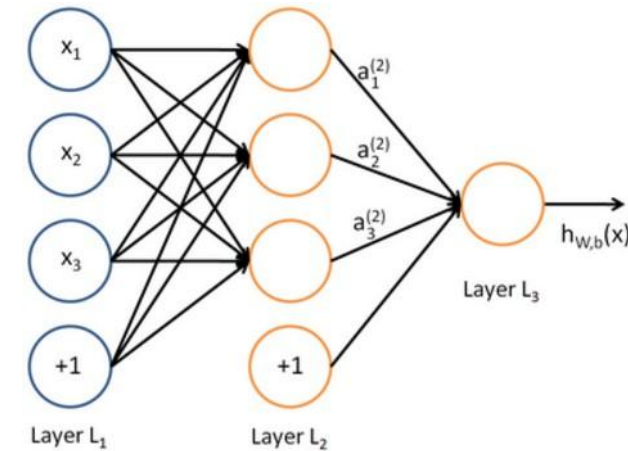
L_l : the layer l

$W_{ij}^{(l)}$: the weight associated with the connection between unit j in layer l ,
and unit i in layer $l + 1$

$b_i^{(l)}$: the bias associated with unit i in layer $l + 1$

$a_i^{(l)}$: the activation unit i in layer l

$z_i^{(l)}$: the total weighted sum of inputs to unit i in layer l



$$a_1^{(2)} = f(W_{11}^{(1)}x_1 + W_{12}^{(1)}x_2 + W_{13}^{(1)}x_3 + b_1^{(1)})$$

$$a_2^{(2)} = f(W_{21}^{(1)}x_1 + W_{22}^{(1)}x_2 + W_{23}^{(1)}x_3 + b_2^{(1)})$$

$$a_3^{(2)} = f(W_{31}^{(1)}x_1 + W_{32}^{(1)}x_2 + W_{33}^{(1)}x_3 + b_3^{(1)})$$

$$h_{W,b}(x) = a_1^{(3)} = f(W_{11}^{(2)}a_1^{(2)} + W_{12}^{(2)}a_2^{(2)} + W_{13}^{(2)}a_3^{(2)} + b_1^{(2)})$$

The Case in Neural Network – Forward Propagation

n_l : the number of layers

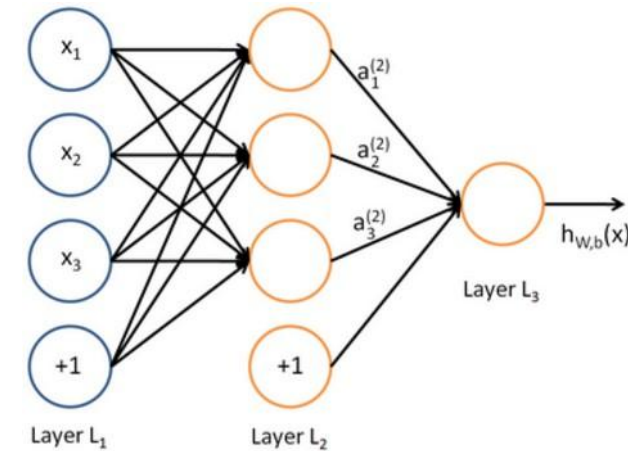
L_l : the layer l

$W_{ij}^{(l)}$: the weight associated with the connection between unit j in layer l ,
and unit i in layer $l + 1$

$b_i^{(l)}$: the bias associated with unit i in layer $l + 1$

$a_i^{(l)}$: the activation unit i in layer l

$z_i^{(l)}$: the total weighted sum of inputs to unit i in layer l



$$a_1^{(2)} = f(\underline{W_{11}^{(1)}x_1 + W_{12}^{(1)}x_2 + W_{13}^{(1)}x_3 + b_1^{(1)}}) \quad z_1^{(2)}$$

$$a_2^{(2)} = f(\underline{W_{21}^{(1)}x_1 + W_{22}^{(1)}x_2 + W_{23}^{(1)}x_3 + b_2^{(1)}}) \quad z_2^{(2)}$$

$$a_3^{(2)} = f(\underline{W_{31}^{(1)}x_1 + W_{32}^{(1)}x_2 + W_{33}^{(1)}x_3 + b_3^{(1)}}) \quad z_3^{(2)}$$

$$h_{W,b}(x) = a_1^{(3)} = f(\underline{W_{11}^{(2)}a_1^{(2)} + W_{12}^{(2)}a_2^{(2)} + W_{13}^{(2)}a_3^{(2)} + b_1^{(2)}}) \quad z_1^{(3)}$$

$$a_i^{(l)} = f(z_i^{(l)})$$

The Case in Neural Network – Forward Propagation

n_l : the number of layers

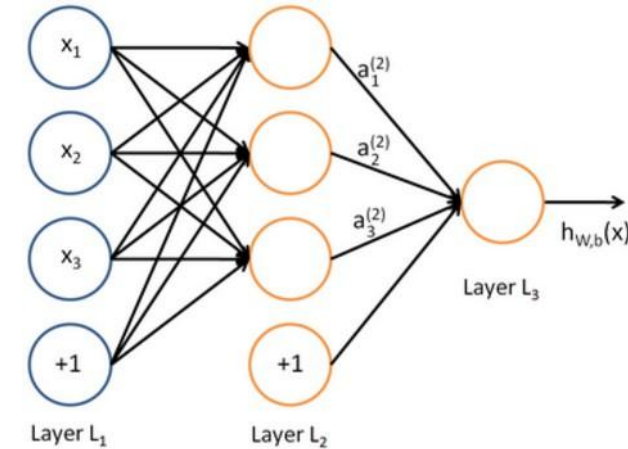
L_l : the layer l

$W_{ij}^{(l)}$: the weight associated with the connection between unit j in layer l ,
and unit i in layer $l + 1$

$b_i^{(l)}$: the bias associated with unit i in layer $l + 1$

$a_i^{(l)}$: the activation unit i in layer l

$z_i^{(l)}$: the total weighted sum of inputs to unit i in layer l

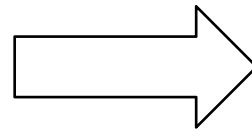


$$z^{(2)} = W^{(1)}x + b^{(1)}$$

$$a^{(2)} = f(z^{(2)})$$

$$z^{(3)} = W^{(2)}a^{(2)} + b^{(2)}$$

$$h_{w,b}(x) = a^{(3)} = f(z^{(3)})$$



$$z^{(l+1)} = W^{(l)}a^{(l)} + b^{(l)}$$

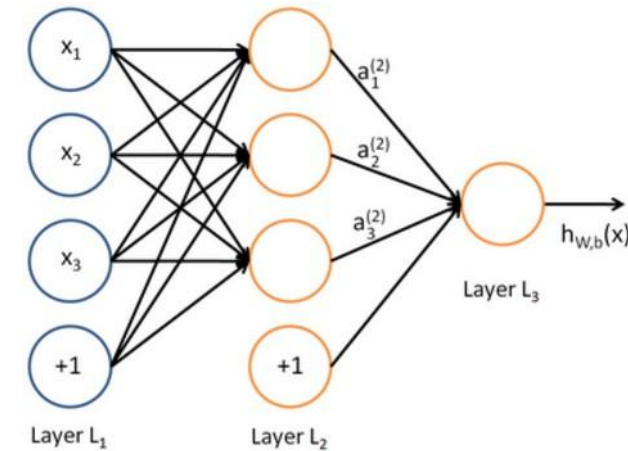
$$a^{(l+1)} = f(z^{(l+1)})$$

The Case in Neural Network – Forward Propagation

- Loss Function:

$$J(W, b; x, y) = \frac{1}{2} \|h_{W,b}(x) - y\|^2.$$

$$\begin{aligned} J(W, b) &= \left[\frac{1}{m} \sum_{i=1}^m J(W, b; x^{(i)}, y^{(i)}) \right] + \frac{\lambda}{2} \sum_{l=1}^{n_l-1} \sum_{i=1}^{s_l} \sum_{j=1}^{s_{l+1}} \left(W_{ji}^{(l)} \right)^2 \\ &= \left[\frac{1}{m} \sum_{i=1}^m \left(\frac{1}{2} \|h_{W,b}(x^{(i)}) - y^{(i)}\|^2 \right) \right] + \frac{\lambda}{2} \sum_{l=1}^{n_l-1} \sum_{i=1}^{s_l} \sum_{j=1}^{s_{l+1}} \left(W_{ji}^{(l)} \right)^2 \end{aligned}$$

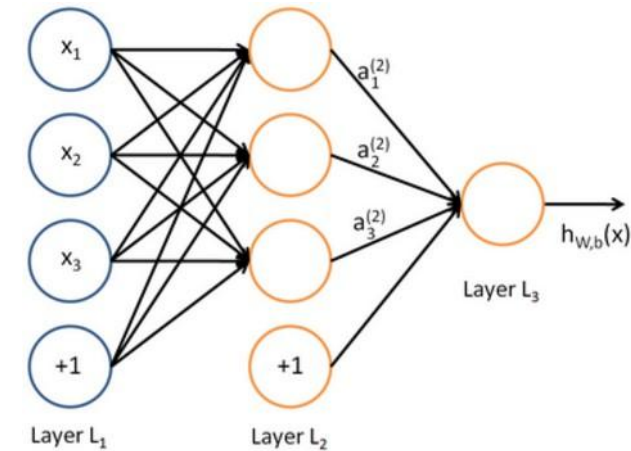


The Case in Neural Network – Backward Propagation

- Gradient Descent:

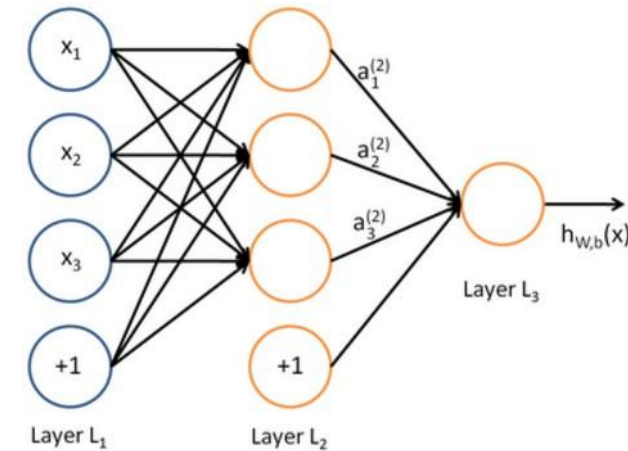
$$J(W, b) = \left[\frac{1}{m} \sum_{i=1}^m J(W, b; x^{(i)}, y^{(i)}) \right] + \underbrace{\frac{\lambda}{2} \sum_{l=1}^{n_l-1} \sum_{i=1}^{s_l} \sum_{j=1}^{s_{l+1}} (W_{ji}^{(l)})^2}_{\text{Regularization Term}}$$
$$= \left[\frac{1}{m} \sum_{i=1}^m \left(\frac{1}{2} \|h_{W,b}(x^{(i)}) - y^{(i)}\|^2 \right) \right] + \frac{\lambda}{2} \sum_{l=1}^{n_l-1} \sum_{i=1}^{s_l} \sum_{j=1}^{s_{l+1}} (W_{ji}^{(l)})^2$$

$$W_{ij}^{(l)} = W_{ij}^{(l)} - \alpha \frac{\partial}{\partial W_{ij}^{(l)}} J(W, b)$$
$$b_i^{(l)} = b_i^{(l)} - \alpha \frac{\partial}{\partial b_i^{(l)}} J(W, b)$$



The Case in Neural Network – Chain Rule

- Compute: $\frac{\partial J}{\partial W_{ij}^{(l)}}$
- Considering:
 - $a_i^{(l+1)} = f(z_i^{(l+1)})$
 - $z_i^{(l+1)} = \sum_{j=1}^{s_l} W_{ij}^{(l)} a_j^{(l)} + b_i^{(l)}$
- Using Chain Rule:

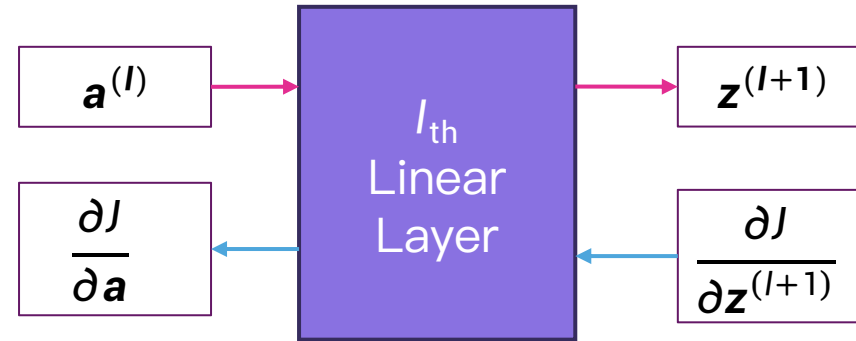


- And we can also get:

$$\frac{\partial J}{\partial W_{ij}^{(l)}} = \frac{\partial J}{\partial z_i^{(l+1)}} \cdot \frac{\partial z_i^{(l+1)}}{\partial W_{ij}^{(l)}} = \frac{\partial J}{\partial z_i^{(l+1)}} \cdot a_j^{(l)} \xrightarrow[\text{Matrix Form}]{} \frac{\partial J}{\partial W^{(l)}} = \mathbf{a}^{(l)} \cdot \frac{\partial J}{\partial \mathbf{z}^{(l+1)}}$$

$$\frac{\partial J}{\partial a_i^{(l)}} = \sum_j \frac{\partial J}{\partial z_j^{(l+1)}} \cdot \frac{\partial z_j^{(l+1)}}{\partial a_i^{(l)}}$$

The Case in Neural Network — A Modularized View



A **Linear Layer** consists of:

1. Parameters: $\mathbf{W}$ & $\mathbf{b}$

2. **Forward Function:**

[INPUT] $\mathbf{a}^{(l)}$

[OUTPUT] $\mathbf{z}^{(l+1)} = \mathbf{W} \mathbf{a}^{(l)} + \mathbf{b}$

[STORE] store $\mathbf{a}^{(l)}$

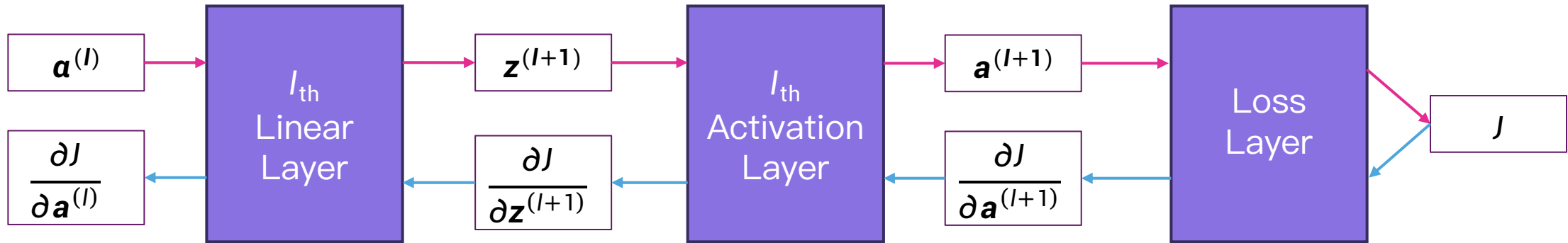
3. **Backward Function:**

[INPUT] $\frac{\partial J}{\partial \mathbf{z}^{(l+1)}}$

[OUTPUT] $\frac{\partial J}{\partial \mathbf{a}^{(l)}} = \left(\frac{\partial \mathbf{z}^{(l+1)}}{\partial \mathbf{a}^{(l)}} \right)^T \frac{\partial J}{\partial \mathbf{z}^{(l+1)}} = \mathbf{W}^T \frac{\partial J}{\partial \mathbf{z}^{(l+1)}}$

[STORE] store $\frac{\partial J}{\partial \mathbf{W}} = \mathbf{a}^{(l)} \cdot \frac{\partial J}{\partial \mathbf{z}^{(l+1)}}$ for later gradient descent

General Cases – A Modularized View



A **Layer** consists of:

1. Some **parameters** for optimization(Optional)

2. **Forward Function:**

compute **output** based on given **input** and **parameters**

store the **input** for computation in **backward function**

3. **Backward Function:**

[INPUT] derivatives to the **output** of **forward function**

[OUTPUT] derivatives to the **input** of **forward function**

[STORE] computes derivatives to the **parameters**(Optional)

Intuition

- Gradients generated when a variable is involved in a computation:

$$z_1 = w_1 a + b_1$$

$$z_2 = w_2 a + b_2$$

$$\frac{\partial J}{\partial a} = \sum_j \frac{\partial J}{\partial z_j} \cdot \frac{\partial z_j}{\partial a}$$

- Another Case:

$$z_1 = w a_1 + b$$

$$z_2 = w a_2 + b$$

$$\frac{\partial J}{\partial w} = \sum_j \frac{\partial J}{\partial z_j} \cdot \frac{\partial z_j}{\partial w}$$

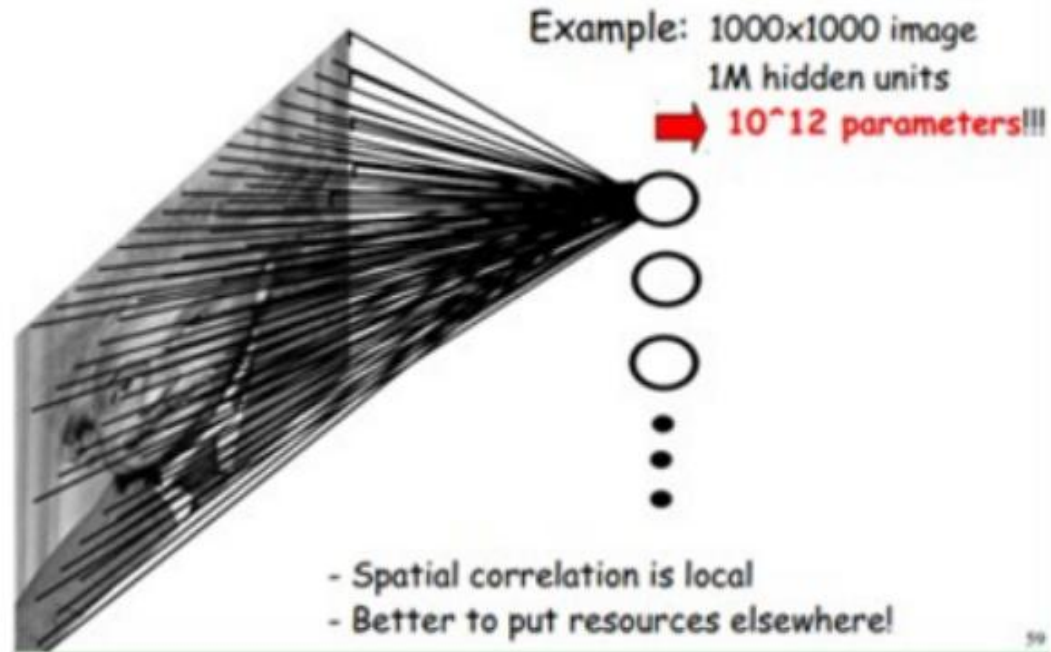
$$\frac{\partial J}{\partial b} = \sum_j \frac{\partial J}{\partial z_j} \cdot \frac{\partial z_j}{\partial b}$$

Content

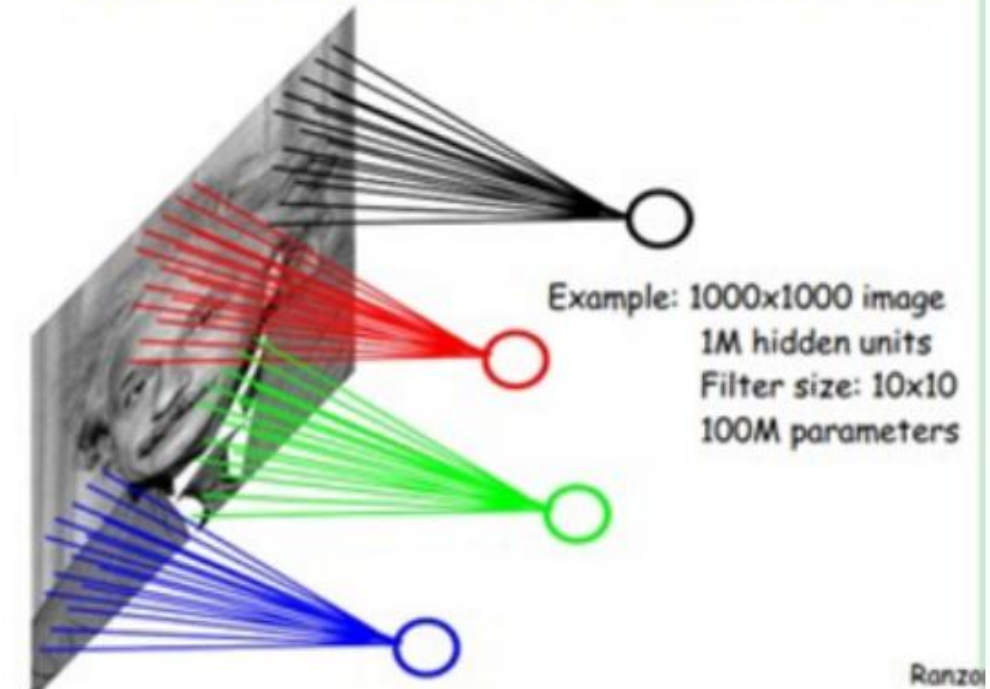
- (Recall) Back Propagation(BP) Algorithm on Neural Network
- **(Recall) Convolutional Network(CNN)**
- BP Algorithm on CNN

Convolutional Neural Network

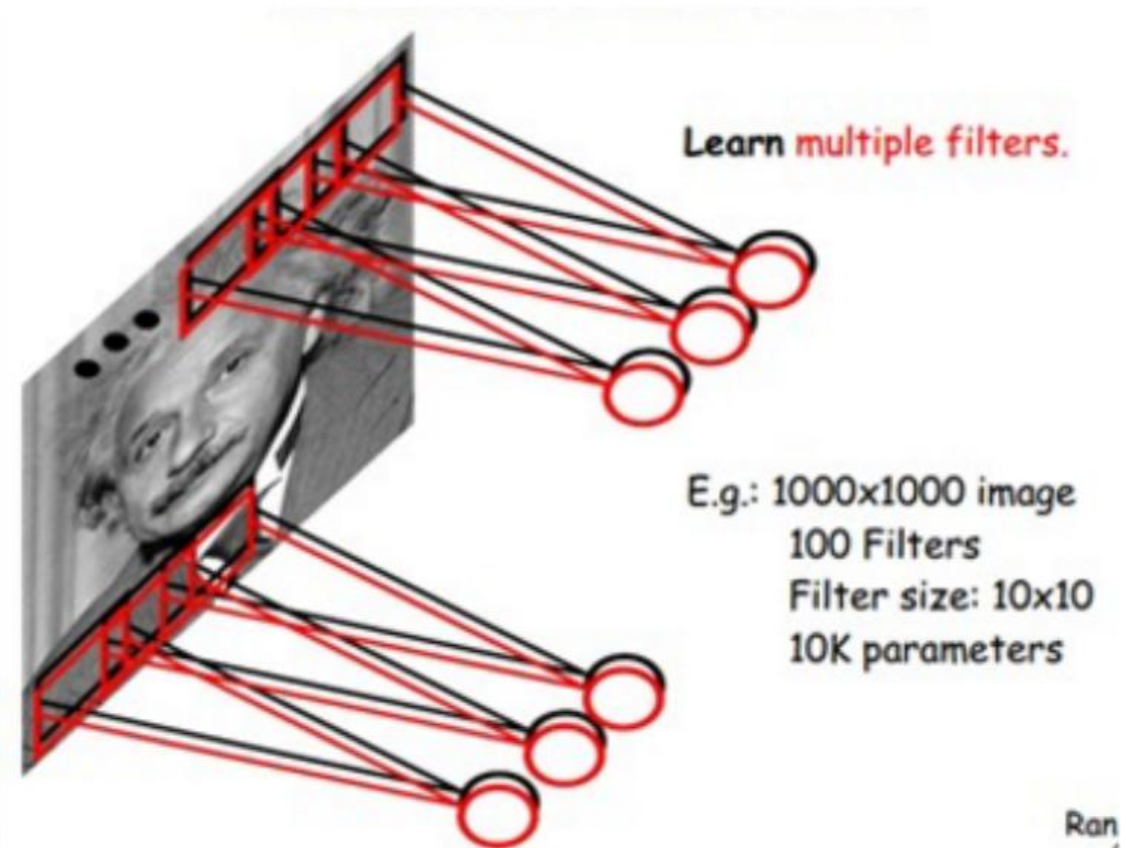
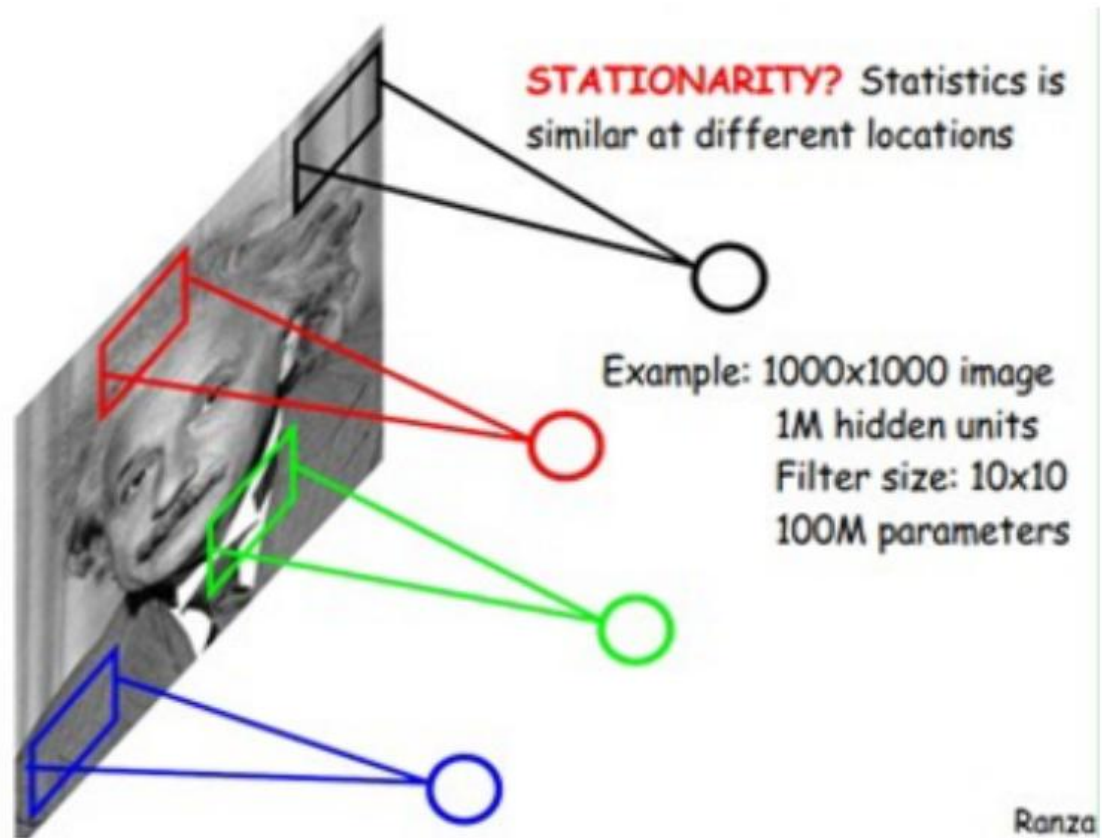
FULLY CONNECTED NEURAL NET



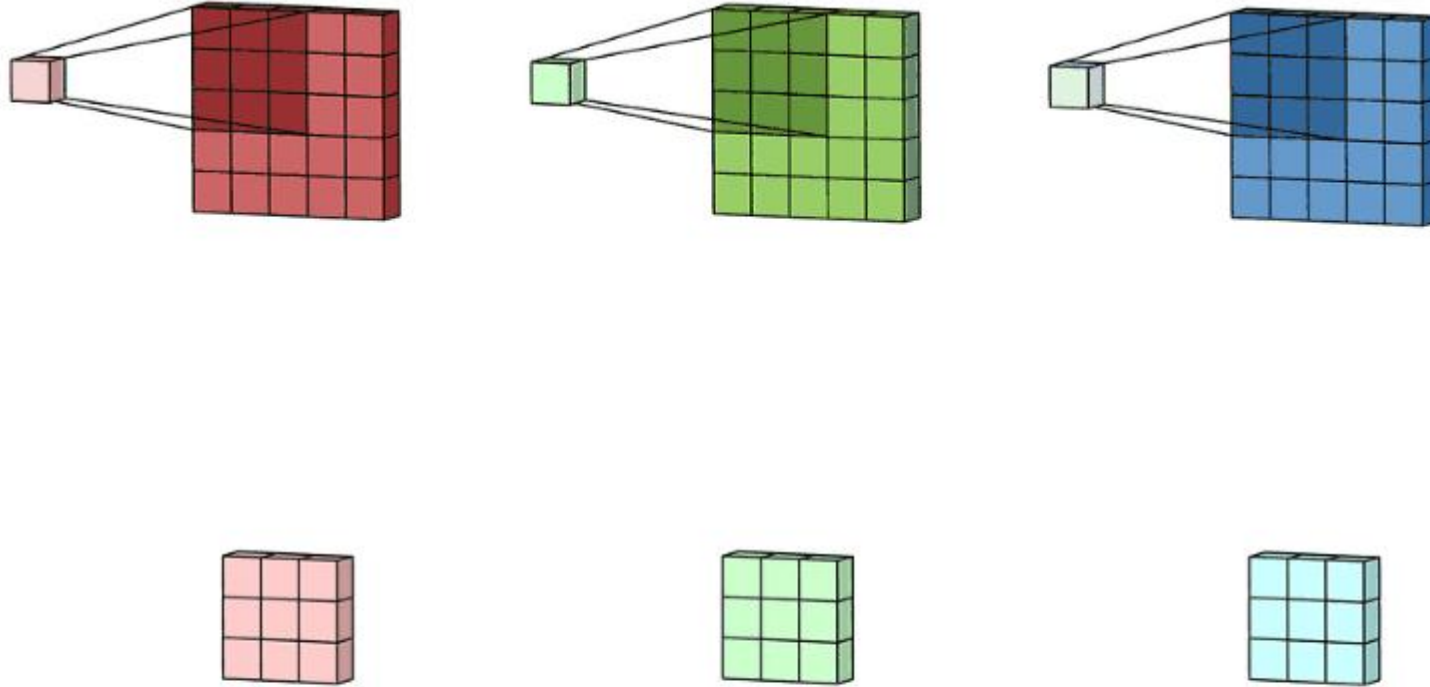
LOCALLY CONNECTED NEURAL NET



Convolutional Neural Network



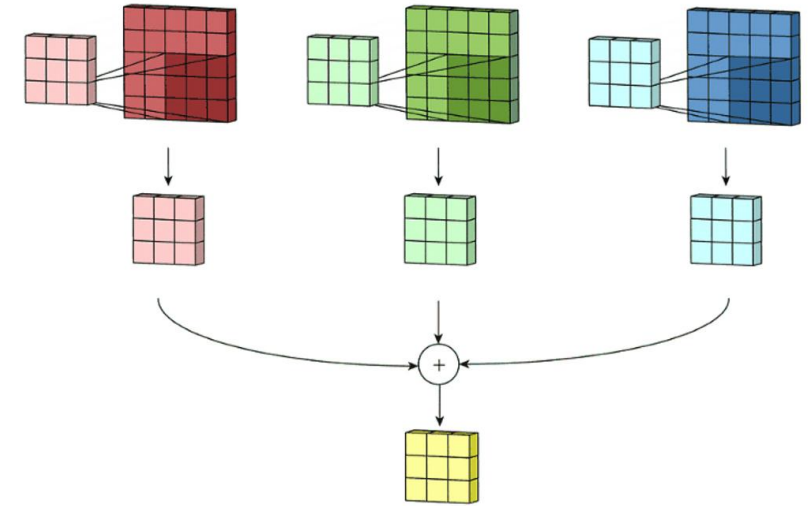
A Convolution Kernel Working on RGB Image



A Convolution Kernel Working on RGB Image

$$C \times H \times W$$

- Input Shape: $3 \times 32 \times 32$
- Kernel Shape: $3 \times 5 \times 5$
- Stride = 1, No Padding
- The Output Shape?

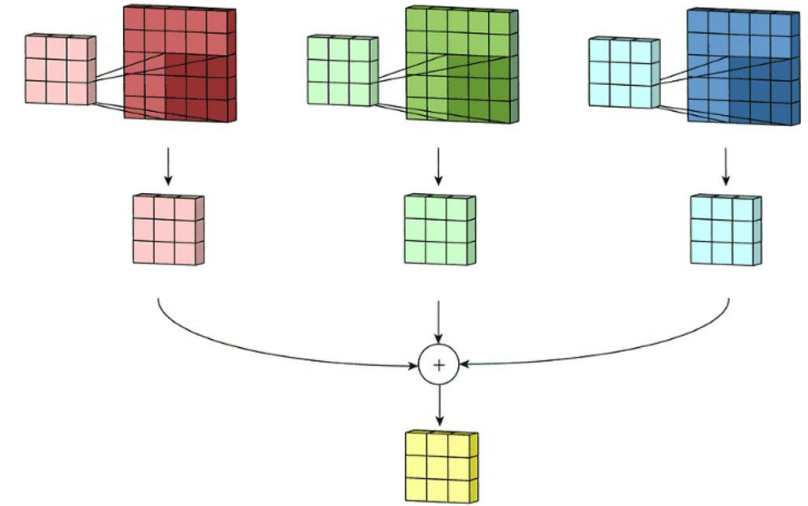


A Convolution Kernel Working on RGB Image

$C \times H \times W$

- Input Shape: $3 \times 32 \times 32$
- Kernel Shape: $3 \times 5 \times 5$
- Stride = 1, No Padding
- The Output Shape?

$1 \times 28 \times 28$



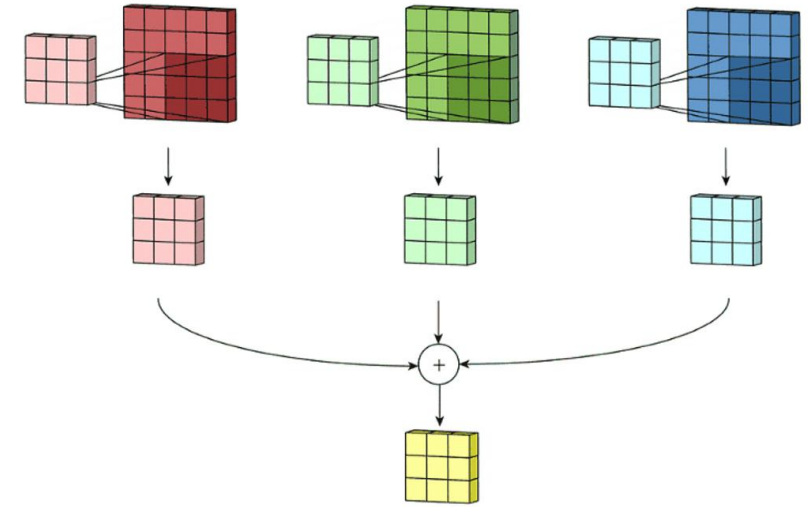
$$H_{l+1} = \frac{(H_l + 2 \times \text{pad_}h - \text{kernel_}h)}{\text{stride_}h} + 1$$

$$W_{l+1} = \frac{(W_l + 2 \times \text{pad_}w - \text{kernel_}w)}{\text{stride_}w} + 1$$

A Convolution Kernel Working on RGB Image

$$C \times H \times W$$

- Input Shape: $3 \times 32 \times 32$
- Kernel Shape: $3 \times 3 \times 3$
- Stride = 2, Padding = 1
- The Output Shape?

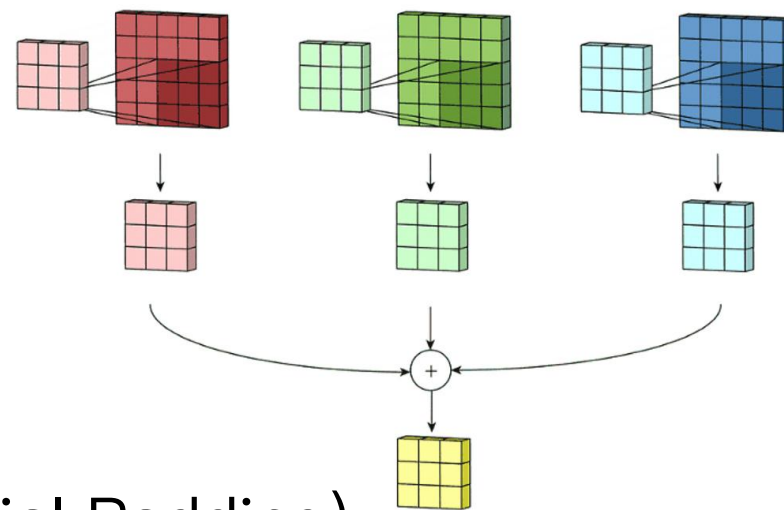


A Convolution Kernel Working on RGB Image

$C \times H \times W$

- Input Shape: $3 \times 32 \times 32$
- Kernel Shape: $3 \times 3 \times 3$
- Stride = 2, Padding = 1
- The Output Shape?

$1 \times 15.5 \times 15.5$? (Flooring, or Special Padding)

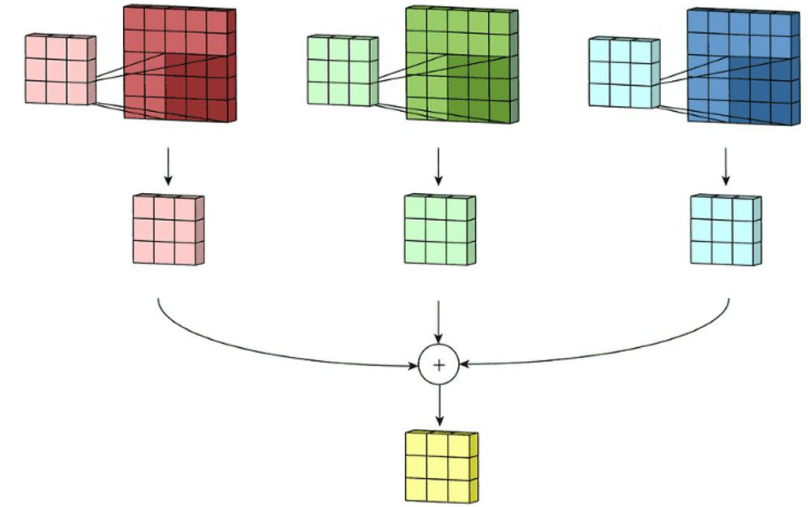


Try it on your own using PyTorch or MindSpore.

$$H_{l+1} = \frac{(H_l + 2 \times \text{pad_}h - \text{kernel_}h)}{\text{stride_}h} + 1 \quad W_{l+1} = \frac{(W_l + 2 \times \text{pad_}w - \text{kernel_}w)}{\text{stride_}w} + 1$$

General Cases — Multiple Kernels

- Input Shape: $C_{in} \times H_{in} \times W_{in}$
- Kernel Shape: $C_{out} \times C_{in} \times H \times W$
- Bias Shape: C_{out}
- Stride S , Padding P
- Output Shape: $C_{out} \times H_{out} \times W_{out}$



Content

- (Recall) Back Propagation(BP) Algorithm on Neural Network
- (Recall) Convolutional Network(CNN)
- **BP Algorithm on CNN**

One Operation Case

- Considering One Operation of One Kernel

$$y_{local} = W * X_{local} + b \quad (\text{逐元素乘后相加})$$

This generates the following gradients:

$$\frac{\partial J}{\partial X_{local}} = \frac{\partial J}{\partial y_{local}} \times \frac{\partial y_{local}}{\partial X_{local}} = \frac{\partial J}{\partial y_{local}} \times W$$

$$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial y_{local}} \times \frac{\partial y_{local}}{\partial W} = \frac{\partial J}{\partial y_{local}} \times X_{local}$$

$$\frac{\partial J}{\partial b} = \frac{\partial J}{\partial y_{local}} \times \frac{\partial y_{local}}{\partial b} = \frac{\partial J}{\partial y_{local}}$$

X_{local}					y_{local}		
3 ₀	3 ₁	2 ₂	1	0	12.0	12.0	17.0
0 ₂	0 ₂	1 ₀	3	1	10.0	17.0	19.0
3 ₀	1 ₁	2 ₂	2	3	9.0	6.0	14.0
2	0	0	2	2			
2	0	0	0	1			

One Operation Case

- Considering One Operation of One Kernel

$$y_{local} = W * X_{local} + b \quad (\text{逐元素乘后相加})$$

This generates the following gradients:

$$\frac{\partial J}{\partial X_{local}} = \frac{\partial J}{\partial y_{local}} \times \frac{\partial y_{local}}{\partial X_{local}} = \frac{\partial J}{\partial y_{local}} \times W$$

$$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial y_{local}} \times \frac{\partial y_{local}}{\partial W} = \frac{\partial J}{\partial y_{local}} \times X_{local}$$

$$\frac{\partial J}{\partial b} = \frac{\partial J}{\partial y_{local}} \times \frac{\partial y_{local}}{\partial b} = \frac{\partial J}{\partial y_{local}}$$

X_{local}					y_{local}		
3 ₀	3 ₁	2 ₂	1	0	12.0	12.0	17.0
0 ₂	0 ₂	1 ₀	3	1	10.0	17.0	19.0
3 ₀	1 ₁	2 ₂	2	3	9.0	6.0	14.0
2	0	0	2	2			
2	0	0	0	1			

Recall the Intuition

- Gradients generated when a variable is involved in a computation:

$$z_1 = w_1 a + b_1$$

$$z_2 = w_2 a + b_2$$

$$\frac{\partial J}{\partial a} = \sum_j \frac{\partial J}{\partial z_j} \cdot \frac{\partial z_j}{\partial a}$$

- Another Case:

$$z_1 = w a_1 + b$$

$$z_2 = w a_2 + b$$

$$\frac{\partial J}{\partial w} = \sum_j \frac{\partial J}{\partial z_j} \cdot \frac{\partial z_j}{\partial w}$$

$$\frac{\partial J}{\partial b} = \sum_j \frac{\partial J}{\partial z_j} \cdot \frac{\partial z_j}{\partial b}$$

Full Operation Case — Derivatives to Parameters

- Considering One Operation of One Kernel

$$y_{local} = W * X_{local} + b \quad (\text{逐元素乘后相加})$$

This generates the following gradients:

$$\frac{\partial J}{\partial W} = \sum_{local} \frac{\partial J}{\partial y_{local}} \times \frac{\partial y_{local}}{\partial W} = \sum_{local} \frac{\partial J}{\partial y_{local}} \times X_{local}$$

$$\frac{\partial J}{\partial b} = \sum_{local} \frac{\partial J}{\partial y_{local}} \times \frac{\partial y_{local}}{\partial b} = \sum_{local} \frac{\partial J}{\partial y_{local}}$$

What about $\frac{\partial J}{\partial X}$?

X_{local}

3 ₀	3 ₁	2 ₂	1	0
0 ₂	0 ₂	1 ₀	3	1
3 ₀	1 ₁	2 ₂	2	3
2	0	0	2	2
2	0	0	0	1

y_{local}

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

Recall the Intuition

- Gradients generated when a variable is involved in a computation:

$$z_1 = w_1 a + b_1$$

$$z_2 = w_2 a + b_2$$

$$\frac{\partial J}{\partial a} = \sum_j \frac{\partial J}{\partial z_j} \cdot \frac{\partial z_j}{\partial a}$$

- Another Case:

$$z_1 = w a_1 + b$$

$$z_2 = w a_2 + b$$

$$\frac{\partial J}{\partial w} = \sum_j \frac{\partial J}{\partial z_j} \cdot \frac{\partial z_j}{\partial w}$$

$$\frac{\partial J}{\partial b} = \sum_j \frac{\partial J}{\partial z_j} \cdot \frac{\partial z_j}{\partial b}$$

How Many Results in the green matrix
does the Purple Square contribute to?

3_0	3_1	2_2	1	0
0_2	0_2	1_0	3	1
3_0	1_1	2_2	2	3
2	0	0	2	2
2	0	0	0	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

3 ₀	3 ₁	2 ₂	1	0
0 ₂	0 ₂	1 ₀	3	1
3 ₀	1 ₁	2 ₂	2	3
2	0	0	2	2
2	0	0	0	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

3	3 ₀	2 ₁	1 ₂	0
0	0 ₂	1 ₂	3 ₀	1
3	1 ₀	2 ₁	2 ₂	3
2	0	0	2	2
2	0	0	0	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

3	3	2	1	0
0 ₀	0 ₁	1 ₂	3	1
3 ₂	1 ₂	2 ₀	2	3
2 ₀	0 ₁	0 ₂	2	2
2	0	0	0	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

3	3	2	1	0
0	0 ₀	1 ₁	3 ₂	1
3	1 ₂	2 ₂	2 ₀	3
2	0 ₀	0 ₁	2 ₂	2
2	0	0	0	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

4 Results, 4 Gradients $\frac{\partial J}{\partial X_{local}}$ to accumulate

$$\frac{\partial J}{\partial X_{local}}$$

3 ₀	3 ₁	2 ₂	1	0
0 ₂	0 ₂	1 ₀	3	1
3 ₀	1 ₁	2 ₂	2	3
2	0	0	2	2
2	0	0	0	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

3	3 ₀	2 ₁	1 ₂	0
0	0 ₂	1 ₂	3 ₀	1
3	1 ₀	2 ₁	2 ₂	3
2	0	0	2	2
2	0	0	0	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

$$\frac{\partial J}{\partial X_{local}}$$

3	3	2	1	0
0 ₀	0 ₁	1 ₂	3	1
3 ₂	1 ₂	2 ₀	2	3
2 ₀	0 ₁	0 ₂	2	2
2	0	0	0	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

3	3	2	1	0
0	0 ₀	1 ₁	3 ₂	1
3	1 ₂	2 ₂	2 ₀	3
2	0 ₀	0 ₁	2 ₂	2
2	0	0	0	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

4 Results, 4 Gradients $\frac{\partial J}{\partial X_{local}}$ to accumulate

 The location of $\frac{\partial J}{\partial X_{local}}$

Full Operation Case – Derivatives to Parameters

Add all $\frac{\partial J}{\partial X_{local}}$ to their own location

We can get $\frac{\partial J}{\partial X}$

3	3	2	1	0
0	0 ₀	1 ₁	3 ₂	1
3	1 ₂	2 ₂	2 ₀	3
2	0 ₀	0 ₁	2 ₂	2
2	0	0	0	1

3	3	2	1	0
0	0 ₀	1 ₁	3 ₂	1
3	1 ₂	2 ₂	2 ₀	3
2	0 ₀	0 ₁	2 ₂	2
2	0	0	0	1

3	3	2	1	0
0	0 ₀	1 ₁	3 ₂	1
3	1 ₂	2 ₂	2 ₀	3
2	0 ₀	0 ₁	2 ₂	2
2	0	0	0	1

3	3	2	1	0
0	0 ₀	1 ₁	3 ₂	1
3	1 ₂	2 ₂	2 ₀	3
2	0 ₀	0 ₁	2 ₂	2
2	0	0	0	1

3	3	2	1	0
0	0 ₀	1 ₁	3 ₂	1
3	1 ₂	2 ₂	2 ₀	3
2	0 ₀	0 ₁	2 ₂	2
2	0	0	0	1

3	3	2	1	0
0	0 ₀	1 ₁	3 ₂	1
3	1 ₂	2 ₂	2 ₀	3
2	0 ₀	0 ₁	2 ₂	2
2	0	0	0	1

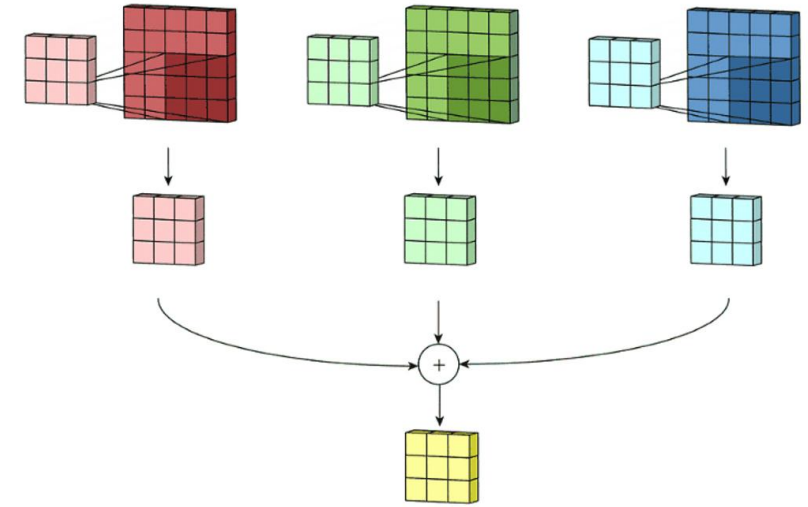
3	3	2	1	0
0	0 ₀	1 ₁	3 ₂	1
3	1 ₂	2 ₂	2 ₀	3
2	0 ₀	0 ₁	2 ₂	2
2	0	0	0	1

3	3	2	1	0
0	0 ₀	1 ₁	3 ₂	1
3	1 ₂	2 ₂	2 ₀	3
2	0 ₀	0 ₁	2 ₂	2
2	0	0	0	1

3	3	2	1	0
0	0 ₀	1 ₁	3 ₂	1
3	1 ₂	2 ₂	2 ₀	3
2	0 ₀	0 ₁	2 ₂	2
2	0	0	0	1

What about Case in reality?

- Input Shape: $C_{in} \times H_{in} \times W_{in}$
- Kernel Shape: $C_{out} \times C_{in} \times H \times W$
- Bias Shape: C_{out}
- Stride S , Padding P
- Output Shape: $C_{out} \times H_{out} \times W_{out}$



Pooling Layer — Simplified Conv

Forward

1	2	3	4
5	6	7	8
9	2	3	5
6	5	1	1

max pooling
→
kernel_size=2
stride=2
pad=0

6	8
9	5

Backward

0	0	0	0
0	1	0	2
3	0	0	4
0	0	0	0

←

1	2
3	4

Pooling Layer — Simplified Conv

Forward

1	2	3	4
5	6	7	8
9	2	3	5
6	5	1	1

avg pooling

kernel_size=2
stride=2
pad=0

7	11
11	5

Backward

0.25	0.25	0.5	0.5
0.25	0.25	0.5	0.5
0.75	0.75	1	1
0.75	0.75	1	1

1	2
3	4

Operators in PyTorch, MindSpore

- Every variable is modeled as a `Tensor`:
 - W, b, X, Y
- When a `Tensor` being operated it will be connected to the computation graph(计算图)
- In a computation graph, each node is a Tensor, and each edge represents an operation
- Grads are automatically computed.

